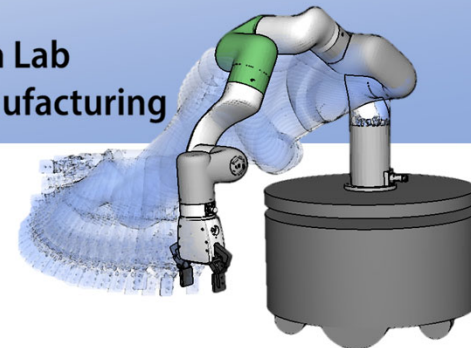


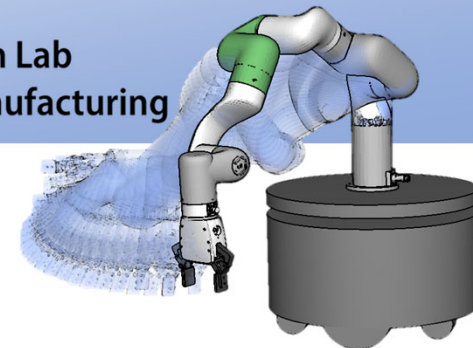


C言語：配列、関数

原田研究室 准教授 万 偉偉

基礎工学部 F522号室

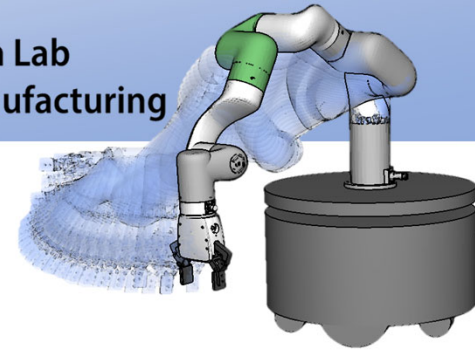
wan@sys.es.osaka-u.ac.jp



復習

- 基本3構造
 - 連接構造、選択構造、反復構造
 - 単一選択、分岐選択、複数選択
 - 前判定反復、後判定反復、カウンタ制御反復
 - フローチャート・NSチャートの図示
- 疑似言語
 - バッカス記法で表現すること

```
<positveinteger> ::= <digit> | <digit_positive><digit_string>  
<digist_positive> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9  
<digit_string> ::= <digit> | <digit><digit_string>  
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```



復習

• 疑似言語

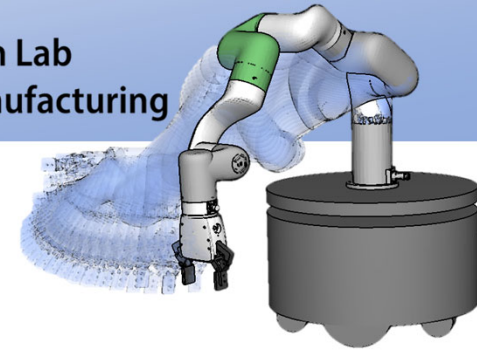
```
if <条件式> then
    <文の列>
else
    <文の列>
endif
```

```
while <条件式> do
    <文の列>
endwhile
```

```
do
    <文の列>
while <条件式>
```

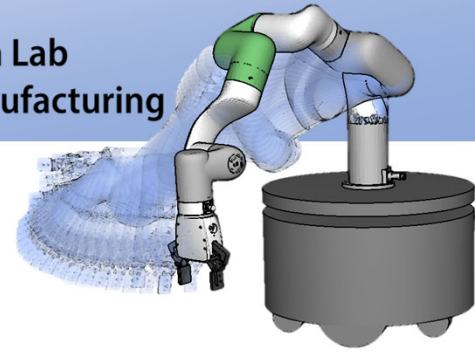
```
for <変数> = <初期値> to <最終値> do
    <文の列>
endfor
```

整数 (int), 実数 (float), 文字 (char), 文字列 (String), 論理値 (boolean)
配列 変数名[]

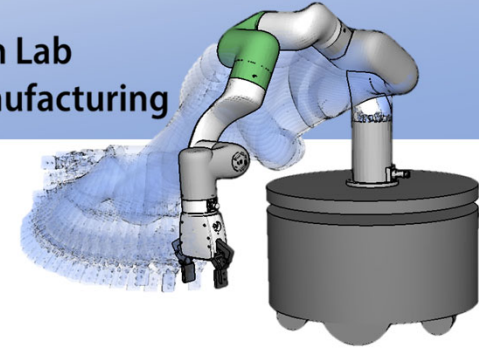


今日の内容

- 配列
 - 宣言、初期化（2次配列、3次配列）
 - 最大値の計算
 - 2次配列の横計、縦計、総合計の計算
- 関数
 - 宣言、仮引数、実引数、呼び出し方法、返り値
 - 局所変数、大域変数
 - 記憶クラス：
 - static 隠す、初期化、メモリ確保
 - extern リンケージ
 - auto 大域変数に付けない
 - リンケージ：複数のファイル
 - 関数間のデータのやり取り
 - 値渡し、参照渡し、大域変数を利用、ポインタの利用

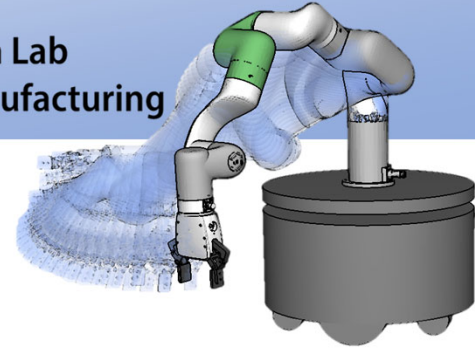


C言語：配列



配列と構造体

- 配列(array)
 - 同じデータ型を持った要素を集めたもの
 - 配列の要素は, 添字 (インデックス,index) により参照可能
- 構造体(structure)
 - 異なるデータ型を意味的にひとまとまりにしたもの



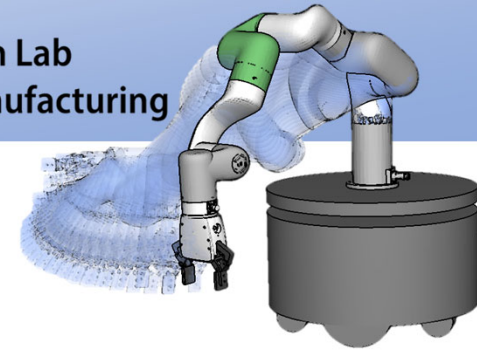
C言語の配列

- データ型 配列名 [要素数];
 - 指定されたデータ型が要素数分だけ連続した格納領域の宣言

例) `int data[4];`

0～3番までの4個の `int` 領域が確保される。

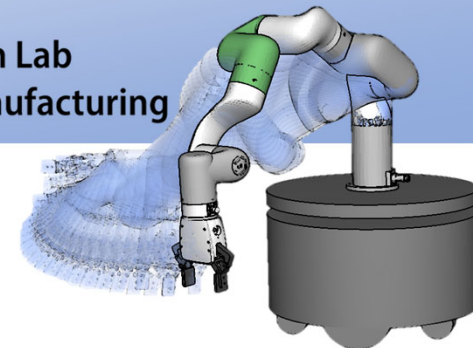
<code>data[0]</code>	<code>data[1]</code>	<code>data[2]</code>	<code>data[3]</code>
<code>int</code>	<code>int</code>	<code>int</code>	<code>int</code>



配列の初期化

- データ型 配列名[要素数] = {定数, 定数,...,定数}

例) `int data[4] = { 7, 4, 3, 10 };`



例題 1

- 例えば，最大値を求めるには
 - 最大値の条件：
 - 他のすべてのデータより大きい（事後条件）

$$\exists m \forall x [m \geq x]$$

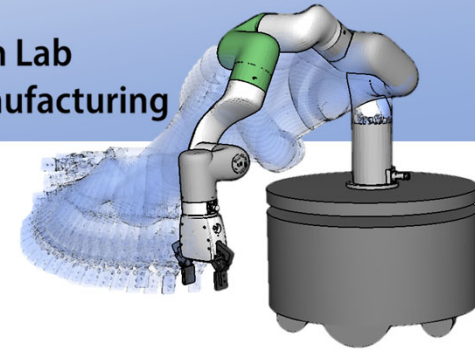
- すべてのデータと比較が必要

疑似言語記述：

```

1) m ← data[0]
2) for I=1 to N-1 do
    If m < data[I] then m ← data[I]
  endIf
endFor
  
```

繰り返し
 最大値の条件
 最大値になるよう修正



配列データの最大値計算

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    int data[10] = { 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 };
```

```
    int i, m, n = 10;
```

配列の要素数

```
        m = data[0];
```

```
        for (i=1; i<n; ++i)
```

```
            if ( m < data[i]) m = data[i];
```

```
        printf( "maximum = %d¥n", m );
```

```
        return 0;
```

```
    }
```

疑似言語をC言語
に変換した部分

m ← data[0]

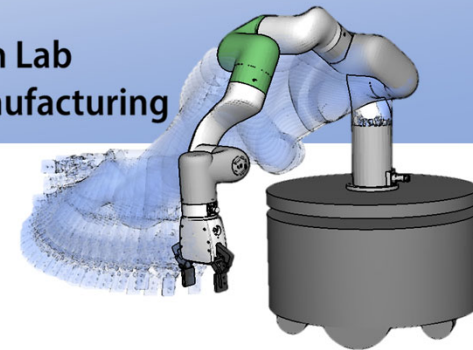
for I=1 to N-1 do

If m < data[I] then

m ← data[I]

endIf

endFor



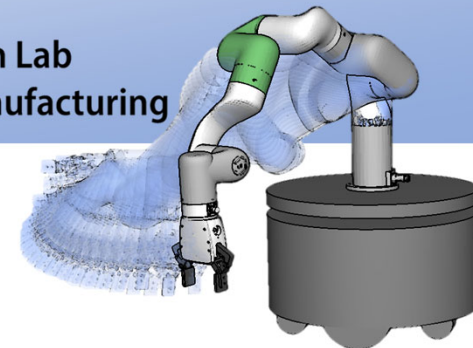
配列と次元 (2次元配列, 3次元配列)

- 次元とは, 配列に含まれる要素を識別するために必要な添え字の個数のこと

データ型	配列名[要素数]	; 1次元配列
データ型	配列名[要素数][要素数]	; 2次元配列
データ型	配列名[要素数][要素数][要素数]	; 3次元配列

例) 3人で4回トランプをした場合の得点表

整数型2次元配列で表現 `int score[3][4];`



2次元配列

行 \ 列	0	1	2	3
0	(0,0)	(0,1)	(0,2)	(0,3)
1	(1,0)	(1,1)	(1,2)	(1,3)
2	(2,0)	(2,1)	(2,2)	(2,3)

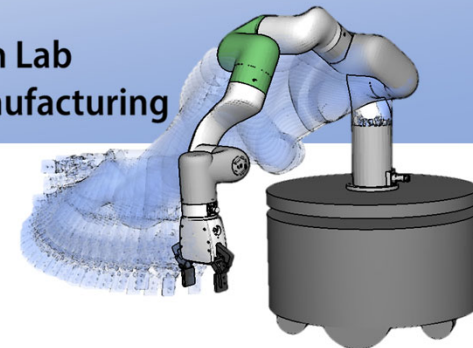
列にゲームの回数，行にプレイヤー

メモリ（計算機）の中ではどんな形で格納？

```
int score[3][4];
```

[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	...	[2][2]	[2][3]

低位アドレス —————> 高位アドレス



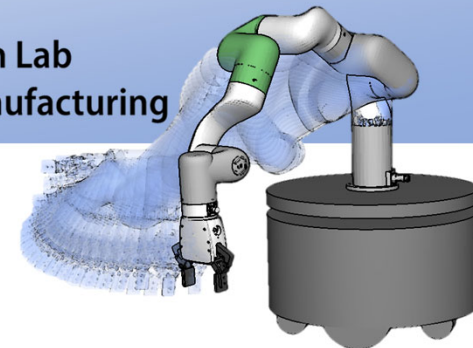
2次元配列の初期値設定

データ型 配列[要素数][要素数] =
 { {定数, 定数, ... , 定数},
 {定数, 定数, ... , 定数},

 {定数, 定数, ... , 定数} };

```
int score[3][4] = {
    { 7, 45, 3, 10 },
    { 11, -3, 8, 17 },
    { 90, 41, 32, 100 } };
```

[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	...	[2][2]	[2][3]
7	45	3	10	11	...	32	100



例題 2

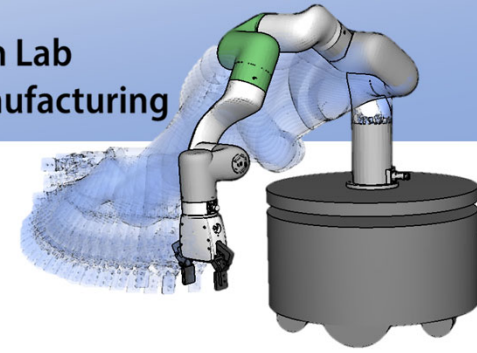
- 4 行 4 列のデータを入力し，縦計，横計，総合計を求め，表形式で出力せよ

○問題の分析

縦計，横計のデータ領域も必要であるから，
5 行 5 列の配列を用意する．

基本方針)

- 1) 16 個のデータを配列に入力
- 2) 縦計，横計，総合計を求める
- 3) 結果の表形式での表示



疑似コーディング

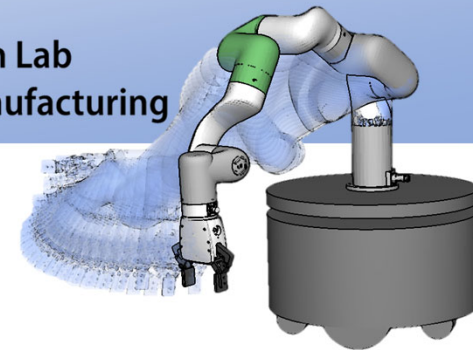
- 16個のデータを配列に入力
 - 4行4列のデータ入力であり，各行は同じ形式としてよいから，まず1行分の入力を4回行う

1-1) for $i=0$ to 3 do

1-2) 1行分 ($\text{data}[i]$) を入力

1行には4列分のデータがある

故に $\text{data}[i][0]$, $\text{data}[i][1]$, $\text{data}[i][2]$, $\text{data}[i][3]$
にデータが格納されるようにする



疑似コーディング(2)

- 1 行分を入力する
 - 行の場合と同様に列も 4 回繰り返し

1-1) for i=0 to 3 do

1-2) 1 行分 (data[i]) を入力

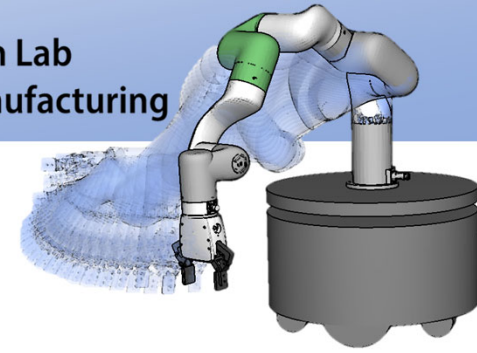


1-1) for i=0 to 3 do

1-2) for j=0 to 3 do

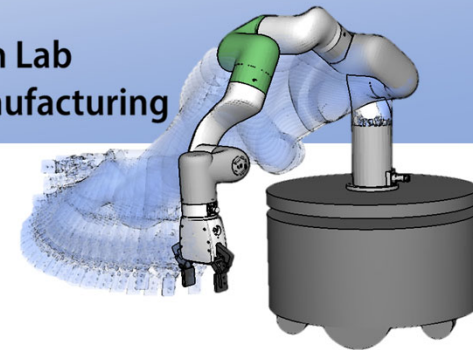
1-3) 1 つのデータ (data[i][j]) を入力する

繰り返し回数を覚えている
変数は違うものを利用



疑似コーディング(3)

- 縦計, 横計, 総合計を求める
 - <縦計の計算>
 - <横計の計算>
 - 縦計の場合と行と列を入れ替える



疑似コーディング(4)

- <総合計の計算>

- 縦計か横計が得られれば計算できる
- 事後条件
 - 総合計 = Σ 縦計 = Σ 横計
- 縦計の総和と, 横計の総和を求め比較
- 正しい総合計ならば `data[4][4]` に格納

1-1) `tate` $\leftarrow$ 0, `yoko` $\leftarrow$ 0

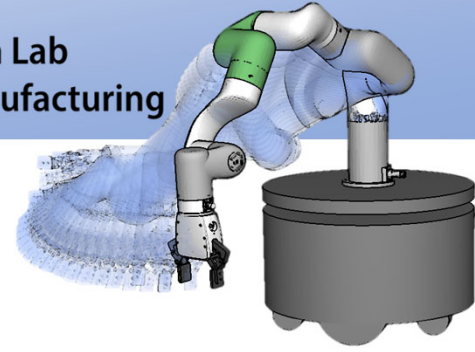
1-2) for `i`=0 to 3 do

1-3) `tate` $\leftarrow$ `tate` + `data[4][i]`

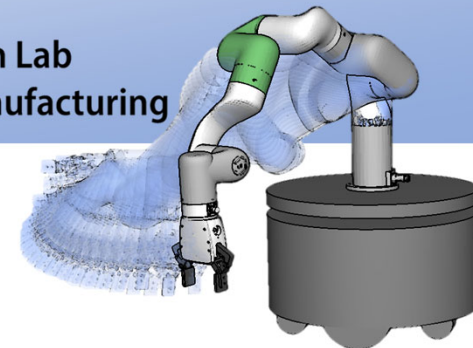
`yoko` $\leftarrow$ `yoko` + `data[i][4]`

1-4) if `tate` = `yoko` then `data[4][4]` $\leftarrow$ `tate`

もちろん `tate` = `yoko` なので `yoko` でもよい



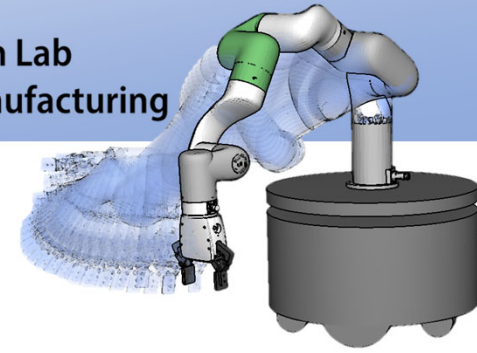
C言語：関数



サブプログラム

- 大きなプログラムでは、扱う値は異なっても、**同一の処理が異なる位置**に現れることがある。
- このような場合、**同一の処理を別途記述**し、元のプログラムからその**処理部分**を呼び出すようにできると便利

独立した処理部分：サブプログラム，モジュール
サブルーチン，関数などと呼ばれる



Cプログラムの特徴

- 関数の集合体

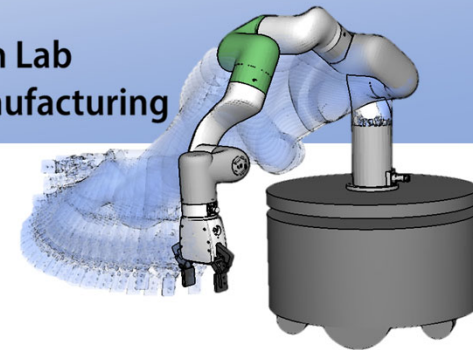
- main関数, printf関数, scanf関数など

返値のデータ型 関数名 (仮引数の型と変数宣言の並び)

```
{  
    内部変数の宣言 ;  
    実行部分 (処理内容) ;  
}
```

関数の定義

仮引数：呼び出し側から与えられるデータを受け取る変数



main関数

- プログラムが起動した時に最初に呼ばれる関数
- 通常, プログラムはオペレーティングシステムから起動される.
- main関数の返値は起動したオペレーティングシステムに渡される.

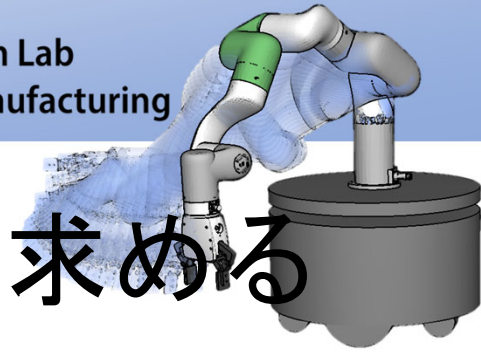
```
int main(int argc, char *argv) {
```

処理内容 ;

return 0; → プログラムが正常終了した時は

} 0 を返す

OSから渡されるデータ



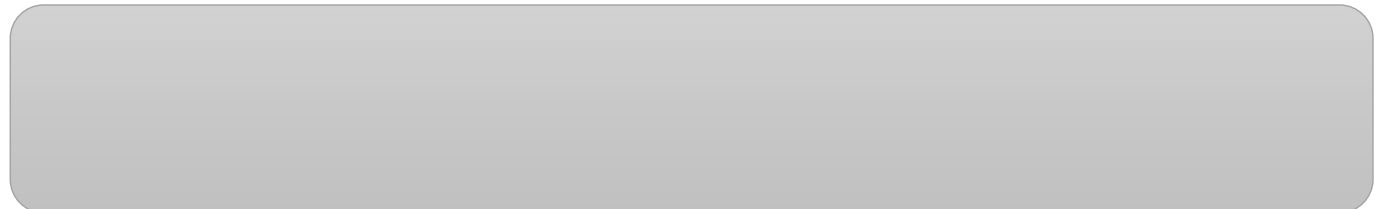
例) 関数 $F(x) = 2x^2 + 6x + 4$ を求める

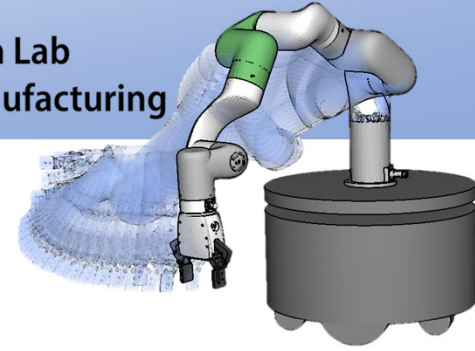
```
double F(double x) → 関数Fの宣言, 仮引数xのデータ型宣言
{
    → 関数Fの定義開始
    double f;      → double型変数fの宣言
    f=2*x*x+6*x+4; → 関数Fの計算
    return f;      → 計算結果を呼び出し側に戻す
} → 関数Fの定義終了
```

関数（サブプログラム等を含む）の利点：

- ・ 関数の外部仕様（事前条件, 事後条件）だけを知っておけば良い
- ・ 関数の内部仕様（実装方法や不変条件）は知らなくて良い

例えば： sin関数や cos関数などの実装方法を知っている人は？





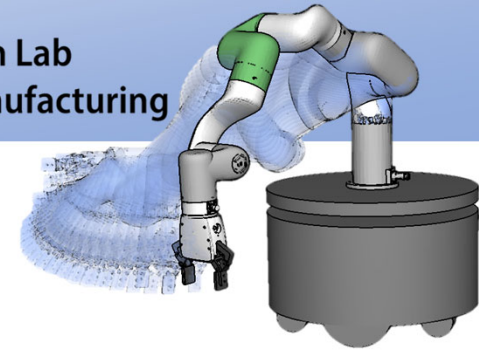
関数の呼び出し方

関数名（実引数の並び）；

実引数：呼び出し側（例えば，main関数）で関数に与えたデータのこと．

実引数の並び：実引数をカンマ（，）で区切ったもの

例：F(x,4);

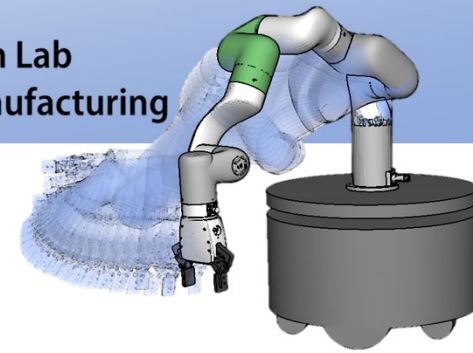


関数の返値

- 呼び出した関数から戻ってくる時に値を返すことができる.

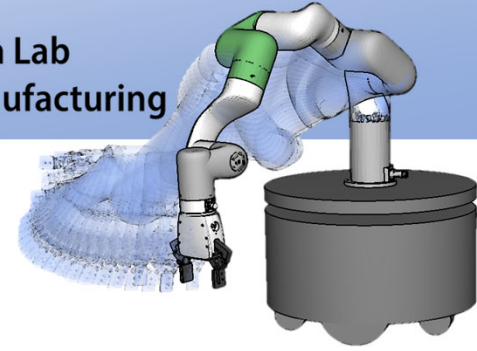
```
return 式;
```

値を返さない場合（型関数）
「式」は省略可能（`return`自体も省略可能）



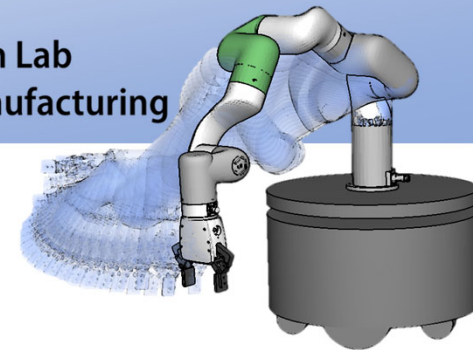
引数と局所変数

- 関数を呼び出す側→実引数
- 呼び出される関数→仮引数
 - 実引数，仮引数との間では
 - 引数の個数
 - 並びの順序
 - データ型 一致しなければならない
 - 実引数→式であってても良い
 - 仮引数→変数
 - 仮引数と実引数の変数名は一致する必要はない
(データ型は一致する必要がある)



引数と局所変数

- 変数にはその名前が有効な範囲(scope)が存在する
- 変数の局所性はプログラムの可読性を高めるために利用する



名前の有効範囲(scope)

```
#include <stdio.h>
```

```
int F(          /* 関数Fの宣言, これ以降Fは有効 */
    int x, int y) /* 仮引数の宣言, 関数F内で有効 */
```

```
{
    int i, j, f = 0;          /* 整数型変数の宣言 */
    for (i = 0; i < y; ++i) {
        int temp = 0;        /* 整数型変数の宣言 */
        for (j = 0; j < x; ++j)
            temp += j;
        f += temp;
    }
    return f;
}
```

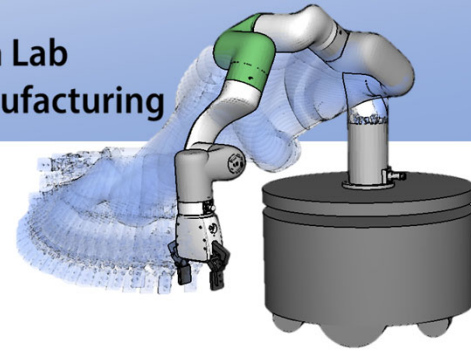
局所変数

tempの有効範囲

```
int main(void)
{
```

```
    int i = 10, j = 4;

    printf( "F(%d,%d) = %d¥n", i, j, F(i,j) );
    printf( "F(%d,%d) = %d¥n", 3, i+j, F(3, i+j) );
}
```



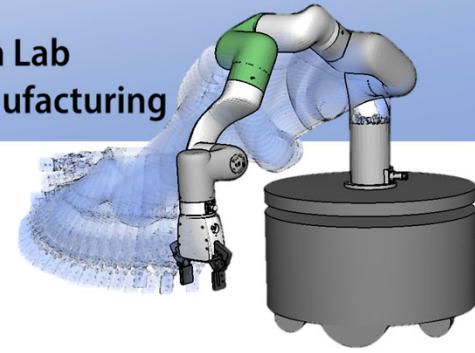
局所変数と大域変数（C言語）

- 局所変数

- 関数内部で定義され，その関数でのみ使用可能
- 複数の関数が同一名称の局所変数を利用しても衝突しない
- { }で囲むことにより、さらに有効範囲を狭くすることも可能

- 大域変数

- 関数外部で定義され，どの関数からでも使用できる
- プログラム中では同一名称の大域変数は一つしかない



局所変数と大域変数の例

```
#include <stdio.h>
```

```
int x=2; ← 大域変数
```

```
void main(){
```

```
    int y; ← 局所変数
```

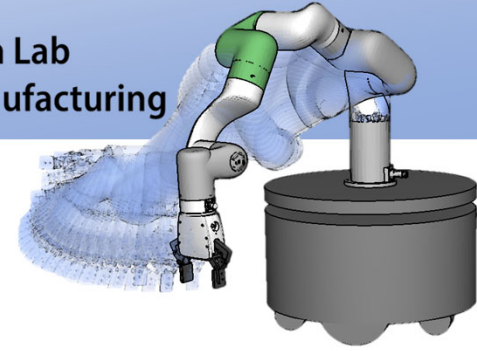
```
    y=call();
```

```
    printf("%d¥n",y);
```

```
}
```

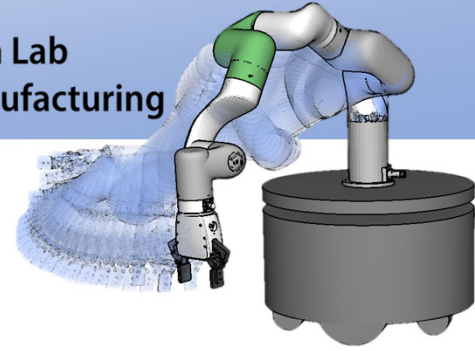
```
int call(){  
    return 2*x;  
}
```

} xは、どこでも参照可
yは、ここでは参照不可



変数宣言と定義

- 宣言：識別子の名前（とか型名とか）を解決するために使う（メモリは確保されない．コンパイラへの指示）
- 定義：その識別子で領域を確保して指定した値を代入する（メモリが確保される）

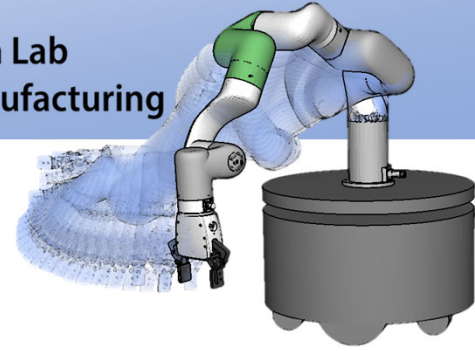


記憶クラス

- 記憶クラス

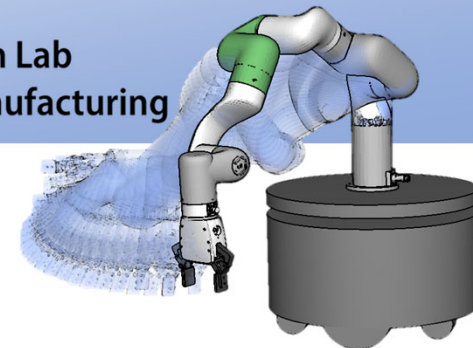
- メモリ上で変数や関数などのオブジェクトを管理する方法の分類

記憶クラス指定子 データ型 変数名 = 初期値;



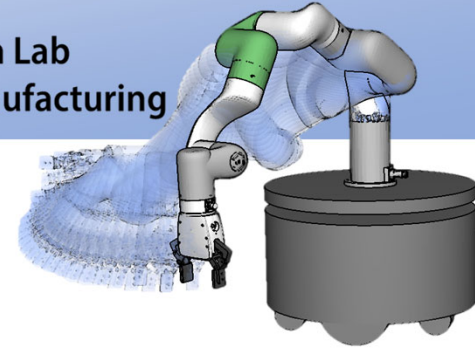
静的変数と自動変数

- 静的変数 (static)
 - プログラム実行中に常に存在して値を保持できる
(常にメモリ領域を確保しておく)
- 自動変数 (auto)
 - 自動変数が宣言されている関数やブロック内に制御が移った時にのみ存在する (メモリ領域を確保する)



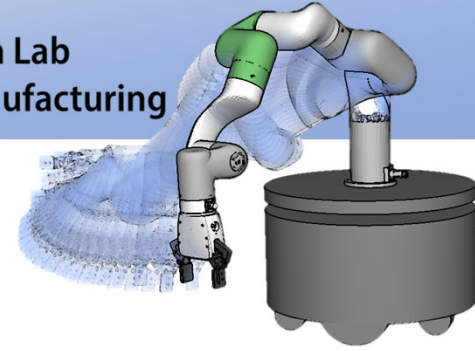
静的変数と自動変数

- 静的変数(static)
 - 局所変数にも大域変数にも付けることができる
 - 局所変数に付けた場合と、大域変数で付けた場合では、意味が大きく異なる
 - 局所変数に付けた場合は、プログラムの実行開始から終了まで、メモリ領域が確保される
 - 大域変数に付けるのは、複数のファイルからプログラムを作成する場合にのみ表れる
 - **staticの付いていない大域変数（外部リンクージを持つ大域変数）はextern記憶クラス指定子により、すべてのファイルに書かれた関数から参照可能**
 - **staticの付いている大域変数（内部リンクージを持つ大域変数）は、その変数が定義されたファイルに書かれた関数からしか参照できない**



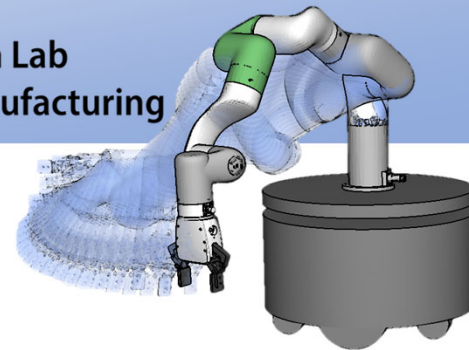
静的変数と自動変数

- 自動変数 (auto)
 - 大域変数に付くことはあり得ない
 - 通常、表記は省略する（局所変数はデフォルトで auto）



静的変数と自動変数

- 大域変数とstatic局所変数
 - 宣言と同時にメモリに割り当てられる
 - 値は0に初期化される
- 自動変数
 - 宣言と同時にメモリに割り当てられる
 - 値は初期化されず、その値は不定



静的変数と自動変数の例

```
void main(){
  int y;
```

← 自動変数 (autoは省略)

```
  y=call(2);
```

```
  printf("%d¥n",y);
```

```
  y=call(2);
```

```
  printf("%d¥n",y);
```

```
}
```

結果は、**両方とも2**

```
int call(int a){
```

```
  int b=1;
```

```
  b=a*b;
```

```
  return b;
```

```
}
```

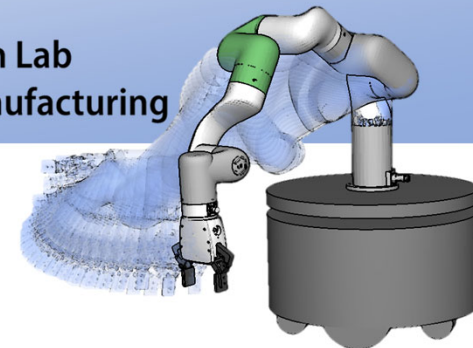
← 自動変数 (autoは省略)

call()の呼び出し
1 回目

call()の呼び出し
2 回目

bの生存期間





静的変数と自動変数の例

```
void main(){
```

```
  int y; ← 自動変数 (autoは省略)
```

```
  y=call(2);
```

```
  printf("%d¥n",y);
```

```
  y=call(2);
```

```
  printf("%d¥n",y);
```

```
}
```

結果は、2と4

```
int call(int a){
```

```
  static int b=1; ← 静的変数 (2回目の呼び出しでは
```

初期化されない！)

```
  b=a*b;
```

```
  return b;
```

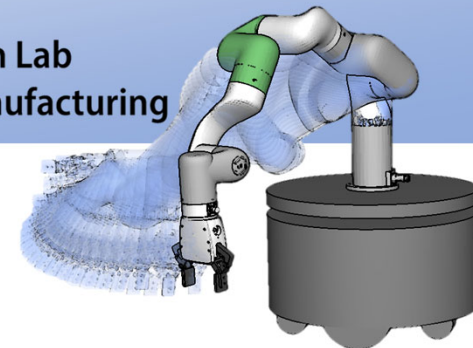
```
}
```

call()の呼び出し
1 回目

call()の呼び出し
2 回目

bの生存期間





静的変数と自動変数の例

```
#include <stdio.h>
```

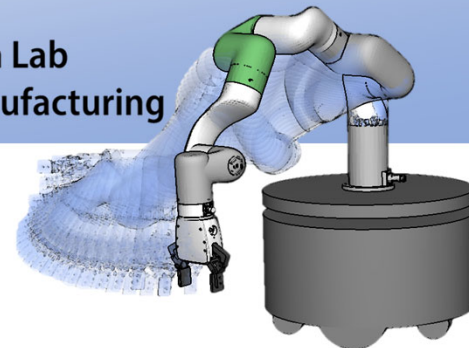
```
int foo(void) {
    static int count = 5;
    return count--;
}
```

```
int count = 1;
```

```
int main(void) {
    printf("global\t\tlocal static\n");
    for(; count <= 5; ++count)
        printf("%d\t\t%d\n", count, foo());
    return 0;
}
```

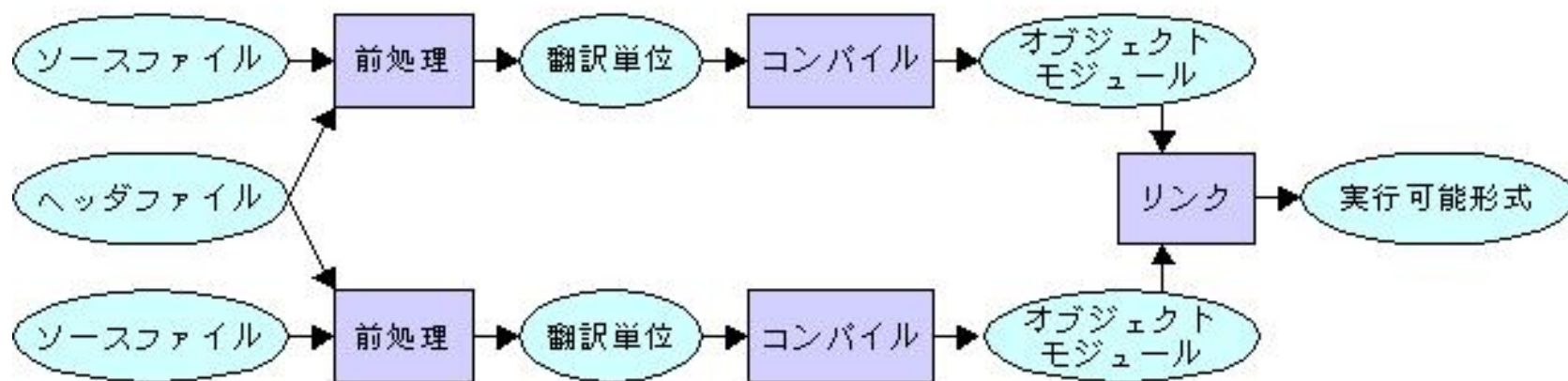
結果

global	local	static
?	?	
?	?	
?	?	
?	?	
?	?	

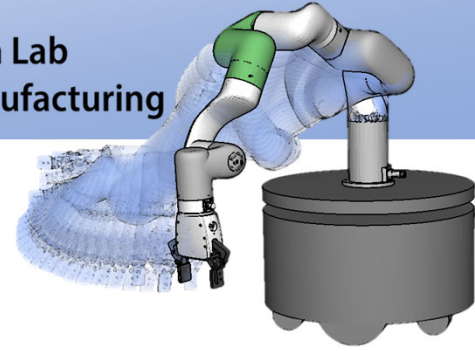


分割コンパイルとリンケージ

大規模なソフト開発では、モジュールごとにソースファイルを作成する



このモジュールは、関数単位となる



分割コンパイルの例

ex1.c

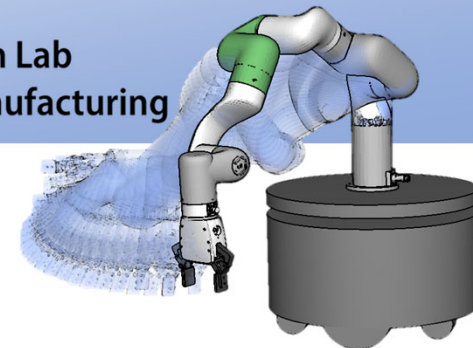
```
#include "ex1.h"
int f(int); // プロトタイプ宣言
void main(void){
    int x=NUMBER;
    printf("%d¥n", f(x));
}
```

ex1.h

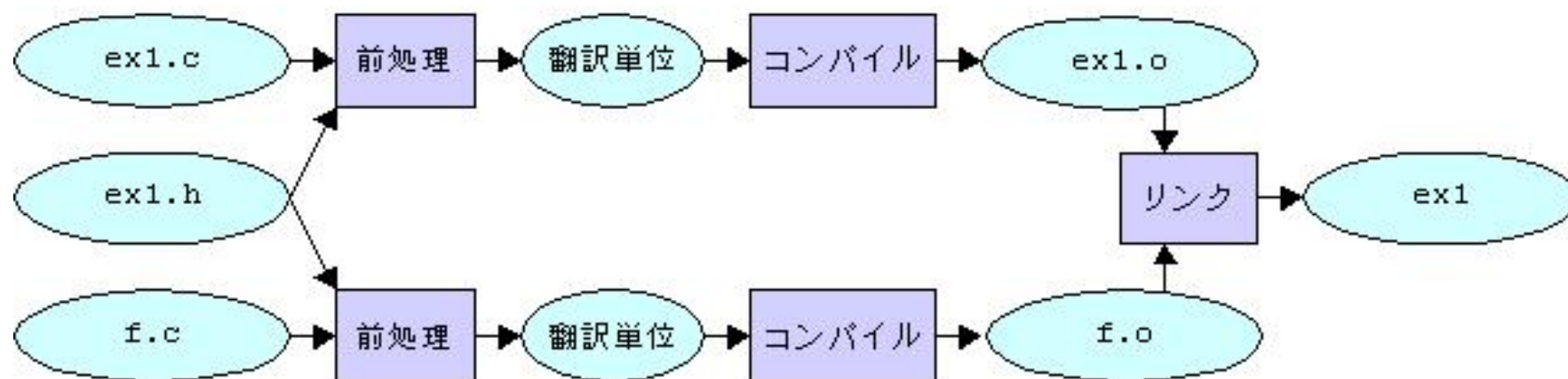
```
#define NUMBER 5
```

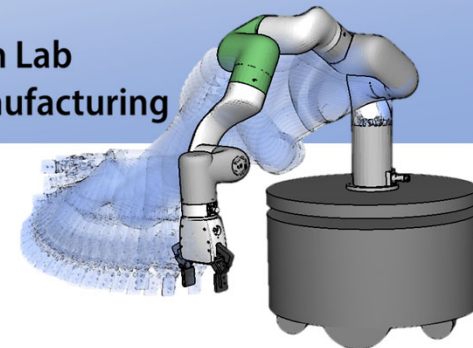
f.c

```
#include "ex1.h"
int f(int x){
    return x*NUMBER;
}
```



分割コンパイルの例





extern指定子

- extern指定子

- 分割コンパイルをする場合で、一つの大域変数を複数の翻訳単位で共通に用いる.

main.c

定義

```
#include <stdio.h>

int x=5; ← 同じメモリ領域を参照

void modify(void);

int main(void)
{
    printf("original: x=%d\n", x);
    modify();
    printf("modified: x=%d\n", x);
    return 0;
}
```

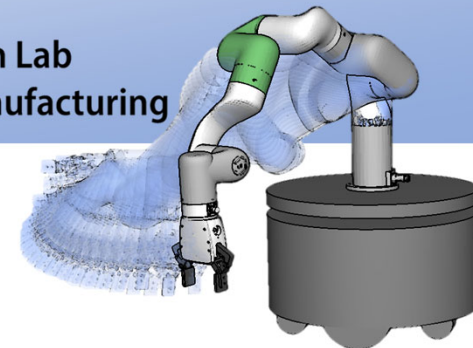
modify.c

```
extern int x;

void modify(void)
{
    x=x*2;
}
```

宣言のみ

同じ変数名の定義を2回以上行うのはおかしいため



staticを付けたら

- 見えなくなる

main.c

```
#include <stdio.h>

static int x =5;

void modify(void);

int main(void)
{
    printf("original: x=%d\n", x);
    modify();
    printf("modified: x=%d\n", x);
    return 0;
}
```

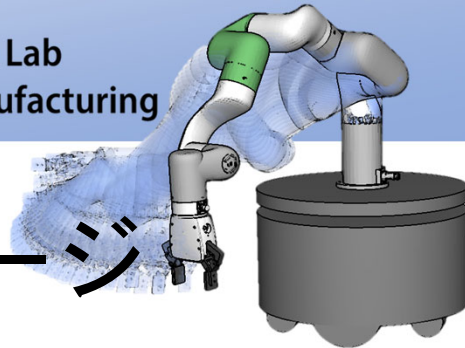
modify.c

```
extern int x;

void modify(void)
{
    x=x*2;
}
```

同じメモリ領域を参照

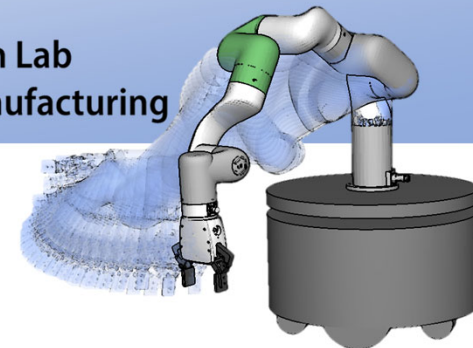
見えなくなる



なぜ見える・見えないか：リンケージ

- 外部リンケージ
 - 翻訳単位（ソースファイル）を超えて同じオブジェクトを参照
- 内部リンケージ
 - 翻訳単位内で同じオブジェクトを参照
 - 翻訳単位が異なれば別のオブジェクトとみなす
- リンケージなし
 - 関数や指定範囲内を超えて同じオブジェクトを参照することはない

C言語のオブジェクト＝変数

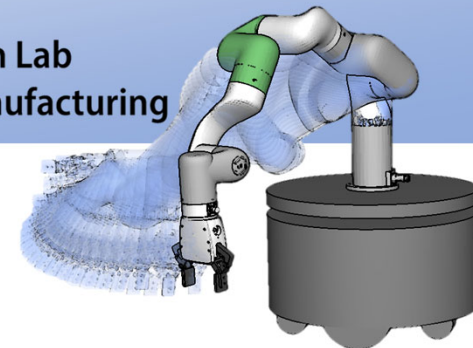


リンケージ（C言語では）

記憶クラス	大域	局所
指定子なし	外部リンケージ	リンケージなし
static	内部リンケージ	リンケージなし
auto	×	リンケージなし

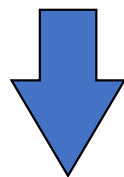
extern指定子

その翻訳単位内の定義の有無	リンケージ
同じ識別子の外部オブジェクトが定義されている	その識別子と同じリンケージ
同じ識別子の外部オブジェクトが定義されていない	外部リンケージ

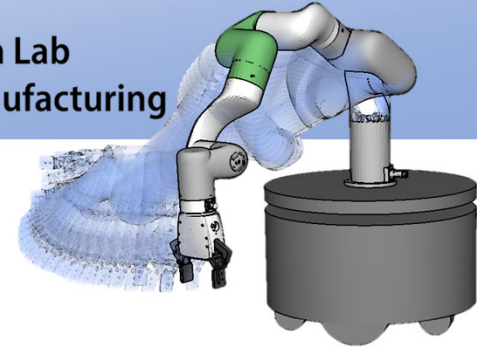


リンケージの例

```
int x;          // 大域変数, 外部リンケージ  
static int y;   // static大域変数, 内部リンケージ
```

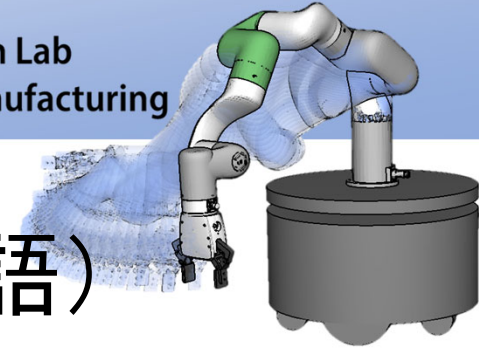


```
int main(void)  
{  
    extern int x; // 外部リンケージ  
    extern int y; // 内部リンケージ  
}
```



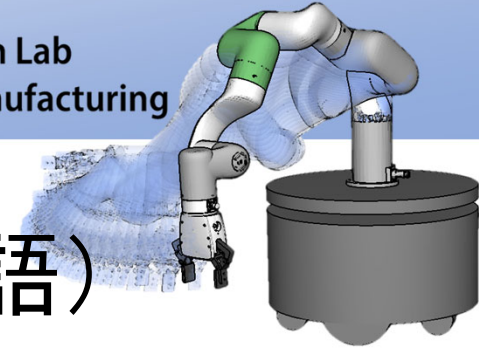
関数（サブルーチン）間の データのやり取り

- 値渡し(Call by Value)
 - ほとんどの言語で利用できる
 - 実引数を評価して、その値を仮引数に代入する
- 参照渡し(Call by Reference)
 - C++, Java, Pascal など
 - 実引数の参照を渡し、仮引数を参照している実体に束縛する



関数間のデータのやり取り（C言語）

- C言語では値渡し(Call by Value)しかできない
- 大域変数を利用する方法もある
- ポインタ型を利用した方法もある

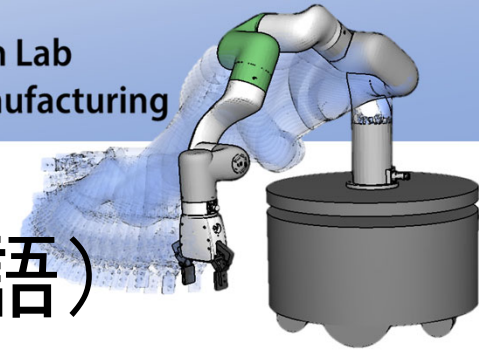


関数間のデータのやり取り（C言語）

- 大域変数の利用

- プログラム全体で存在し，データをやり取りする必要がある場合に利用

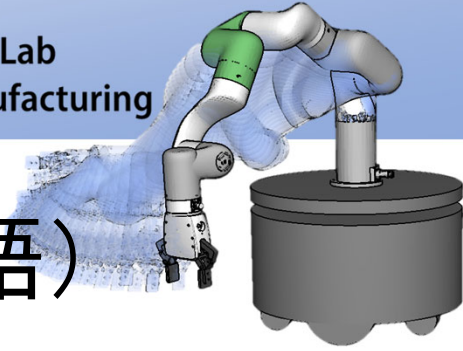
```
int g1;          /* 大域変数の宣言 */
int F(int x) {
    return x+g1; /* 関数内での大域変数の利用 */
}
int main(void) {
    g1 = 10;
    printf( "F(%d) = %d, g1 = %d¥n", 20, F(20), g1 );
    return 0;
}
```



関数間のデータのやり取り（C言語）

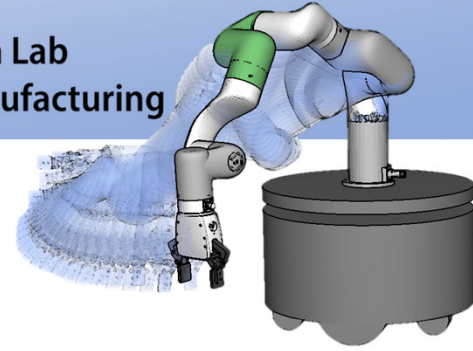
- 大域変数の利用

- 大域変数を利用する関数が多くなると，保守が難しくなる．
- また，可読性も悪くなる
- 他の方法で代替できる時には，大域変数は利用しないほうが良い



関数間のデータのやり取り（C言語）

- ポインタ型の利用
 - 実引数の値を書き換えたいとき
 - 値渡しでは実引数の値はコピーされるので変更できない.
 - 変更したい実引数のポインタを渡す



まとめ

- 配列
 - 宣言、初期化（2次配列、3次配列）
 - 最大値の計算
 - 2次配列の横計、縦計、総合計の計算
- 関数
 - 宣言、仮引数、実引数、呼び出し方法、返り値
 - 局所変数、大域変数
 - 記憶クラス：
 - static 隠す、初期化、メモリ確保
 - auto 大域変数に付けない
 - extern リンケージ
 - リンケージ：外部・内部・なし
 - 関数間のデータのやり取り
 - 値渡し、参照渡し、大域変数を利用、ポインタの利用